

COMP 3030 Assignment Part 2

Kyle Heng

A1955409

Table of Contents

- 1. Overview**
- 2. Part 2A synchronization explanation**
- 3. Part 2A validation**
- 4. Experimental environment and methodology**
- 5. Results**
- 6. Performance Analysis**
- 7. Conclusion**
- 8. Appendix**

1. Overview

This report investigates the correctness and performance of OpenMP parallelisation in a two-dimensional N-body solver. OpenMP allows multiple threads to share the same particle and force data while distributing computational work across CPU cores. Part 2A examines synchronization when multiple OpenMP threads update a shared force array, while Part 2B evaluates how the N-body force algorithm and OpenMP scheduling strategy affect runtime, speedup, efficiency and scalability.

2. Part 2A synchronization explanation

The supplied shared-force solver calculates each particle-pair interaction once and adds equal-and-opposite contributions directly into the shared forces array. When the outer force loop is parallelised, two OpenMP threads may simultaneously update the same `forces[i]` entry, creating a race condition and potentially losing a force contribution.

Both Part 2A implementations use a main OpenMP parallel region with `default(none)`. Shared variables such as the particle array, force array and simulation parameters are explicitly declared shared, while loop variables are private to individual threads. The implicit barrier following the force-calculation `omp for` ensures that all force contributions are complete before any particle position or velocity is updated.

The critical-section implementation protects the four shared force updates for each particle pair with `#pragma omp critical`. This provides simple synchronization but allows only one OpenMP thread at a time to perform force accumulation. The lock-based implementation instead uses one OpenMP lock per particle. For a pair $(part, k)$, the thread locks both corresponding force entries before updating them. Because the reduced pair loop always satisfies $part < k$, locks are acquired in ascending particle-index order, preventing circular wait and deadlock. Unlike the global critical section, two interactions involving completely different particles may update concurrently.

3. Part 2A validation

Both implementations were compared against the supplied sequential reference using identical generated initial conditions. Tests used 1 thread with 4 particles, 2 threads with 4 particles, and 4 threads with 8 particles, with two timesteps of size 0.01.

For every configuration, the displayed positions and velocities from both Part 2A implementations matched the reference at $t = 0.00, 0.01$ and 0.02 after testing. This provides evidence that the synchronization correctly prevents lost force updates and that body updates begin only after the complete force-calculation stage.

4. Experimental environment and methodology

Performance testing was performed on an Apple Macbook Air M1 using gcc Ubuntu 11.4.0. All programs were compiled with OpenMP support and optimisation, with detailed simulation output disabled using `-DNO_OUTPUT`.

All tests used 10,000 particles, 20 timesteps, and a timestep size of 0.01. Thread counts of 1, 4, 8, 16 and 32 were tested. Each implementation/thread configuration was executed five times under the same conditions and the arithmetic mean execution time was reported. Speedup and efficiency was then calculated independently for each implementation using $\text{Speedup} = T_1 / T_p$ and $\text{Efficiency} = \text{Speedup} / p \times 100\%$.

5. Results

Number of Threads	Basic	Reduced Default	Reduced Forces Cyclic	Reduced All Cyclic
1	9.0608	3.6309	3.6203	3.6358
4	2.5669	1.6865	1.0201	1.0256
8	2.1314	1.1316	0.9235	0.9529
16	2.1849	0.9542	0.8804	0.8899
32	2.1287	1.1731	1.0941	1.1358

Table 1. Mean execution time for 10,000 particles and 20 timesteps (in seconds)

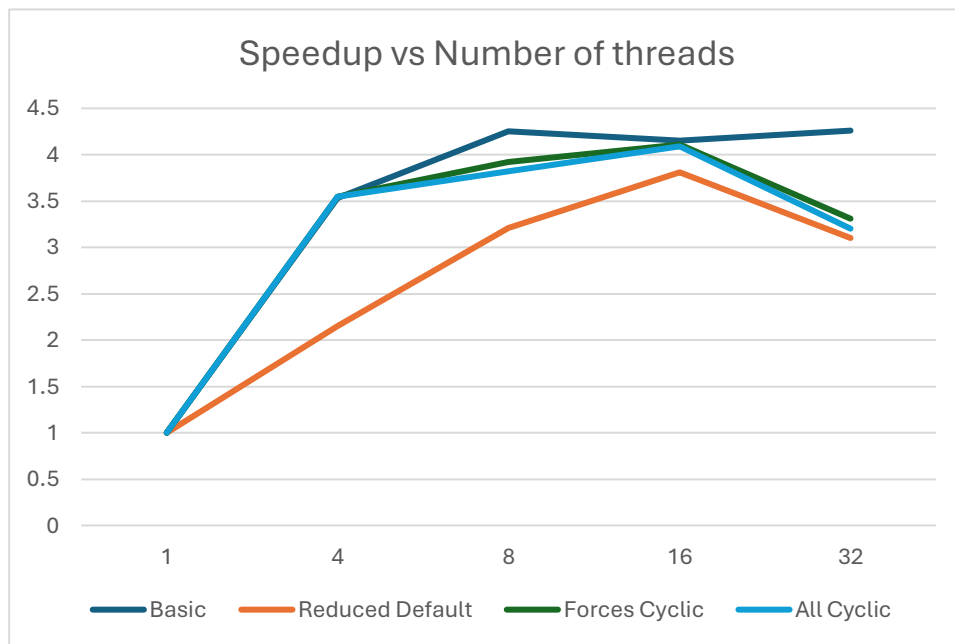


Figure 1. Speedup versus number of OpenMP threads.

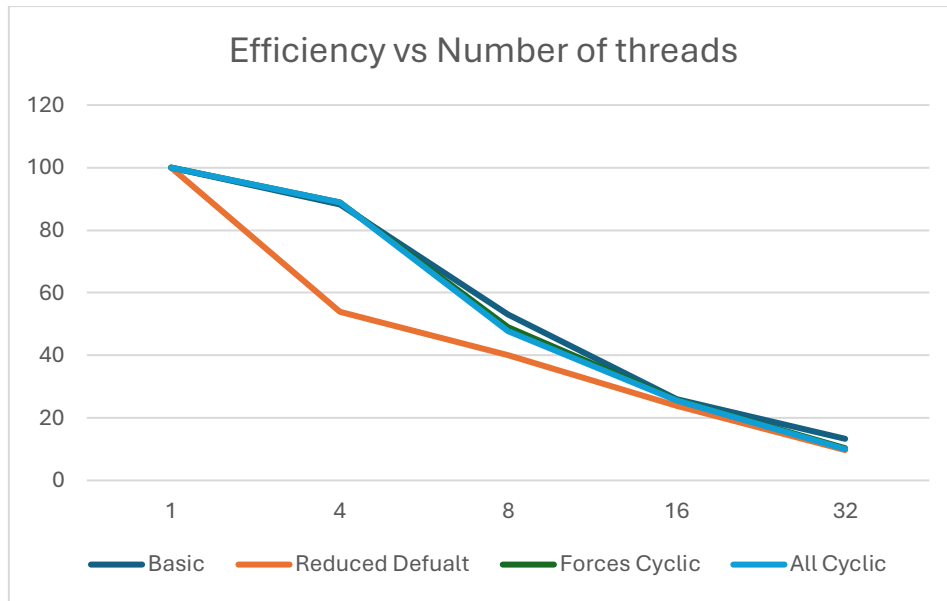


Figure 2. Efficiency versus number of OpenMP threads.

6. Performance analysis

The largest algorithmic difference appears at one thread. Basic required 9.0608s, while all three Reduced implementations required approximately 3.62–3.64s. The Reduced algorithm therefore achieved substantially lower runtime even without parallelism because it exploits force symmetry and calculates each pair only once, whereas Basic recalculates the interaction from the perspective of both particles.

Scheduling had a particularly large effect on the Reduced algorithm. At four threads, Reduced Default required 1.6865s compared with only 1.0201s for Forces Cyclic. The reduced force loop is inherently unbalanced because the iteration for part = 0 performs $n-1$ pair interactions, while progressively larger values of part perform fewer interactions. The supplied cyclic force strategy uses `schedule(static,1)`, distributing these expensive and inexpensive iterations across threads in round-robin order rather than assigning large contiguous blocks. This produced better load balance on the tested system.

Applying cyclic scheduling to every loop did not improve performance further. All Cyclic was consistently similar to, but slightly slower than Forces Cyclic at higher thread counts. The reset, force-combination and particle-update loops have substantially more uniform work per iteration, so cyclic distribution provides little load-balancing benefit and may introduce additional scheduling and cache-access overhead.

Scalability was not linear. Basic improved substantially from one to four threads and reached approximately $4.25\times$ speedup at eight threads, after which its runtime remained at about 2.1s. The Reduced implementations continued improving to approximately 16 threads. Forces Cyclic achieved the lowest observed runtime of 0.8804s at 16 threads, corresponding to a $4.11\times$ speedup over its own one-thread runtime.

All Reduced implementations became slower at 32 threads. Forces Cyclic increased from 0.8804s at 16 threads to 1.0941s at 32 threads. This indicates diminishing returns as scheduling, synchronization, memory/cache traffic and per-thread force-reduction overhead begin to outweigh additional parallelism. The Reduced implementation maintains a separate force array for each OpenMP thread and subsequently combines these arrays; consequently, this reduction overhead also increases with thread count.

Overall, Forces Cyclic provided the best performance on the tested system. It combined the lower computational cost of the Reduced algorithm with cyclic scheduling targeted specifically at the imbalanced force-calculation loop, while avoiding unnecessary cyclic scheduling of the more uniformly distributed loops.

7. Conclusion

The experiments demonstrate that both algorithm design and OpenMP scheduling significantly affect N-body performance. Part 2A showed that synchronization is required for correctness when OpenMP threads update shared force data; however, synchronization granularity can strongly affect performance, with per-particle locks reducing contention compared with a single global critical section.

In Part 2B, all Reduced implementations were substantially faster than Basic because they avoided redundant pairwise force calculations. Cyclic scheduling of the force loop improved workload distribution and produced the best observed performance, while applying cyclic scheduling to all loops provided no additional benefit. The best result was Reduced — Forces Cyclic with 16 OpenMP threads at approximately 0.880 s. Performance declined at 32 threads, showing that additional threads do not guarantee improved runtime once parallel overhead and hardware-resource limits become significant.

8. Appendix

Number of Threads	Basic	Reduced Default	Reduced Forces Cyclic	Reduced All Cyclic
1	1.00	1.00	1.00	1.00
4	3.53	2.15	3.55	3.55
8	4.25	3.21	3.92	3.82
16	4.15	3.81	4.11	4.09
32	4.26	3.10	3.31	3.20

Table 2. Speedup calculations used for Figure 1

Number of Threads	Basic	Reduced Default	Reduced Forces Cyclic	Reduced All Cyclic
1	100.0	100.0	100.0	100.0
4	88.3	53.8	88.8	88.8
8	53.1	40.1	49.0	47.8
16	25.9	23.8	25.7	25.6
32	13.3	9.7	10.3	10.0

Table 3. Efficiency calculations used for Figure 2 (in percentage)